

GJØVIK UNIVERSITY COLLEGE

BACHELOR THESIS

Snowman

A complete, effective and secure rule management system for SNORT®

Authors:
Thomas NYHEIM
Eirik SKOGSTAD
Eigil OBRESTAD

Supervisor:
Slobodan PETROVIC

April 15, 2014

Preface

This is the preface.

Glossary

MD5 A cryptographic hash function producing a 128-bit hash value.

URL Uniform Resource Locator.

Contents

Preface . . . . . **i**
Glossary . . . . . **ii**
Contents . . . . . **iii**
List of Figures . . . . . **v**
List of Tables . . . . . **vi**
1 INTRODUCTION **1**
2 EXISTING SOLUTIONS **2**
3 REQUIREMENTS SPECIFICATION **3**
3.1 Functional Requirements 3
3.1.1 Program Workflow 3
3.1.2 Use Case Diagram 5
3.1.3 High Level Use Cases 5
3.1.4 Detailed Use Cases 9
3.1.5 Domain Model 11
3.2 Supplemental Requirements 11
3.2.1 Functionality 11
3.2.2 Usability 11
3.2.3 Reliability 11
3.2.4 Performance 11
3.2.5 Licensing 11
3.3 Constraints 11
3.3.1 11
3.3.2 11
3.3.3 11
4 DESIGN **12**
5 IMPLEMENTATION **13**
5.1 Frameworks 13
5.2 Concepts of Snort® 13
5.2.1 Rules 13
5.2.2 Generators 13
5.2.3 Reference Types 13
5.2.4 Event and detection filters 13
5.3 Core module 15
5.4 Update module 15
5.4.1 Sources 15
5.4.2 Files 15
5.4.3 Running an update 15
5.4.4 Rule parsing 16
5.4.5 Classification, Generator and Reference Type parsing 17
5.4.6 Logging 17

5.5 Graphical User Interface 17
5.5.1 Structure 17
5.5.2 Design 18
5.5.3 Features 18
5.5.4 Rules/All rules page 18
6 OPTIMIZATION 20
6.1 Update 20
7 SOFTWARE TESTING 21
8 PERFORMANCE TESTING 22
8.1 Environment 22
8.1.1 Test cases 22
9 RESULTS 23
10 ANALYSIS OF RESULTS 25
11 CONCLUSIONS 26
Bibliography 27

List of Figures

1 Program Workflow 3
2 Program Modules 4
3 Flow 14
4 GUI Structure 17
5 Rules/All rules page 18

List of Tables

10 Effects of different filter types. 14
11 Rule options used in this system. 16
12 Test results for Snortmanager. 24

1 INTRODUCTION

Følgende informasjon kan tas med her: - Organisering av rapporten og kort beskrivelse av sammenhengen mellom kapitlene. - Fullstendig definering av oppgaven med utdypning av hva oppgaven går ut på. Eksempelvis presiseres mål, problemområde, omfang og avgrensning. - Henvisning til eventuelle tidligere arbeider om samme emne. - Målgruppe for rapporten, jfr. mål- og mottakeranalysen. - Studentenes faglige bakgrunn. - Prosjektgruppens valgte arbeidsformer, f.eks. samtaler og intervjuer med kilder, reiser, bedriftsbesøk, litteraturstudier, laboratorieforsøk, modellbygging og testing. - Terminologibruk. Er eksempelvis faguttrykk brukt i en spesiell betydning? Innledningen skal ikke inneholde noe om det som er gjort i oppgaven og oppnådde resultater.

2 EXISTING SOLUTIONS

Her tenker jeg at vi tar med hva som fins av programvare fra før og peker påutfordringer i disse løesningene.

3 REQUIREMENTS SPECIFICATION

Many of the requirements for this system stems from functions that already reside in BHTB, which our client wants either reproduced or improved upon. That means the requirments for this project is closely tied to the way BHTB operates and its general workflow. In this coming section we will outline the specifications we've gathered from BHTB and from conversations with the client.

3.1 Functional Requirements

3.1.1 Program Workflow

To develop our requirements, we decided to chart the program workflow in order to better understand what the system is expected to do. Since this is to be a highly user-oriented system, we accomplished this by going over the old system, BHTB, step by step with the client and detailing the functions and various needs for improvement or additional functionality. From this, we developed a workflow chart that details the major system parts.

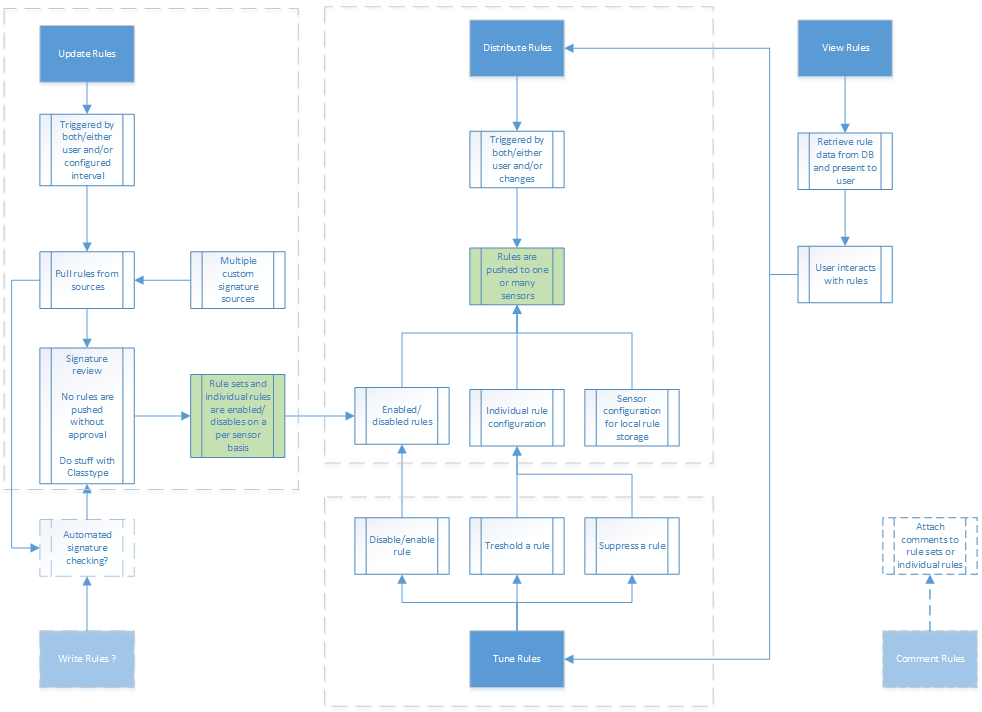


Figure 1: Program Workflow

To explain the workflow chart, the blue boxes are the major system functionalities, the green boxes are the major goals of the system and the arrows show the direction of flow the system should have.

As you can see, if you trace the arrows, all of the program flow ends up with rules being syncronized to the sensors. This makes sense, because that is the whole purpose of the system to begin with.

It's important to note that very early on, we decided that writing rules, automatically check them for errors and commenting them were nice-to-have features that we could implement if we had time to do so. Hence they are greyed out in the drawings.

From this workflow we could outline a rough modular design based on the naturally separate functions of updating rules, tuning them, distributing them and show them. We also added a database module, since most of our program was going to be centered around it.

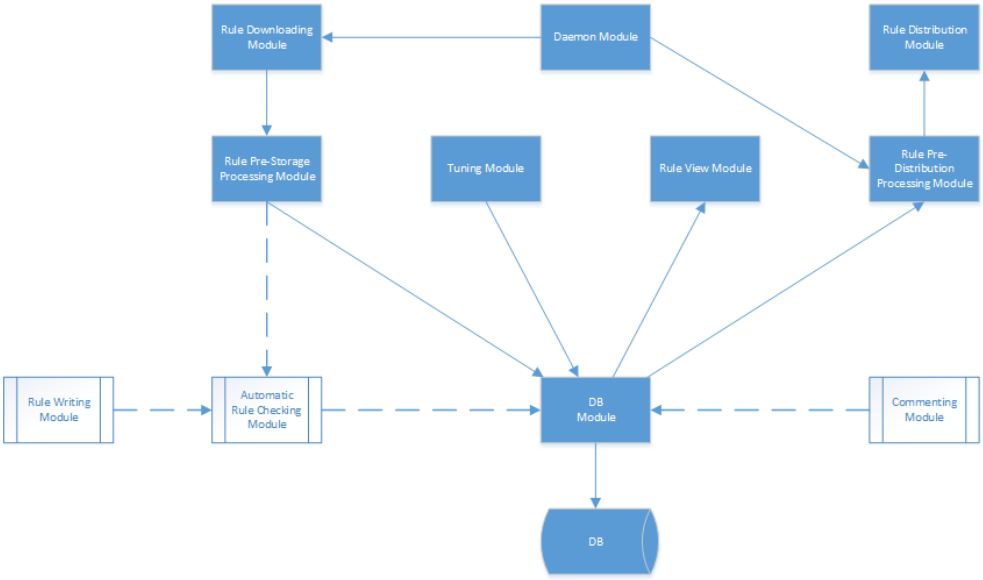


Figure 2: Program Modules

Rule Downloading Module

This module will handle all functionality related to downloading rules, including comparing MD5 checksums.

Rule Pre-Storage Processing Module

This module will essentially take downloaded rules and parse and process them into objects that can be fed into the database.

DB Module

This module will handle all input and output to/from the database.

Rule Pre-Distribution Processing Module

This module will essentially take rules stored in the database and prepare them for distribution to sensors.

Rule Distribution Module

This module will be in charge of distributing rules out to the Snort sensors.

Tuning Module

This module will handle all functionality related to turning rules on or off, thresholding them or suppressing them.

Rule View Module

This module will contain all the functionality needed to display the system status and system functions to the user. It will be the primary way the user interacts with the system.

Daemon Module

This module will run continually on the system to ensure that the system is responsive and to carry out any interval based tasks, such as automatic updates or distributions.

Commenting Module

This module will allow users to attach comments to rules.

Rule Writing Module

This module will handle functionality for writing custom rules and editing existing rules.

Automatic Rule Checking Module

This module will be responsible for verifying rules to ensure that there are no malformed or misconfigured rules that may potentially break or cause a halt on a Snort sensor.

3.1.2 Use Case Diagram

Putt inn nytt use case diagram her!

3.1.3 High Level Use Cases

Looking at the arrows in program workflow figures, it is pretty apparent that the program workflow is heavily centered around the database module, and the major parts of the system revolves around the importing of rules and then distributing them. We therefore began developing our use cases around those major movements.

We've added the most central use cases for this system below, but additional use cases is available in appendix B.

Use case	**Manual rule update**
Primary actor	User
Purpose	Update the central rule database with the newest rules and revisions.
Description	The user commands the server to update the rules, either from a local file or from external sources. When the latter is the case, the server pulls rule files from all the sources specified in the list of sources. In both cases rules are processed and stored in the database.
Preconditions	Either a file or one or more sources must exist.
Postconditions	The central rule database is up to date with the newest rules and revisions.

Use case	**Automatic rule update**
Primary actor	Daemon
Purpose	Update the central rule database with the newest rules and revisions.
Description	At configured time-intervals, the system daemon triggers a rule database update. The server pulls rule files from all the sources specified in the list of sources to be updated at the given interval, processes the rules and then stores them in the database.
Preconditions	One or more sources must exist.
Postconditions	The central rule database is up to date with the newest rules and revisions

Use case	**Manual rule distribution**
Primary actor	User
Purpose	Distribute rules to sensors as per the settings in the database.
Description	The user sends a command to distribute rules to either all or specified sensors. The server then begins the distribute routine, and eventually all sensors will be up to date with the rules and rulesets they should have.
Preconditions	Rules and sensors must exist
Postconditions	Specified sensors contain rules as per settings in the central database.

Use case	**Automatic rule distribution**
Primary actor	Daemon
Purpose	Distribute rules to sensors as per the settings in the database.
Description	Configured at intervals, the system triggers a command to distribute rules to specified sensors. The server then begins the distribute routine, and eventually all sensors will be up to date with the rules and rulesets they should have.
Preconditions	Rules and sensors must exist

Postconditions	Specified sensors contain rules as per settings in the central database.

Use case	**Enable/Disable a rule or ruleset**
Primary actor	User
Purpose	Enable or disable one or more rule or ruleset on one or more sensors.
Description	The user either enables or disables a rule or a ruleset so that the rule is either on or off on certain sensors. This status is then shared with the sensor and the sensor syncronizes as needed.
Preconditions	Rules and/or rulesets must exist.
Postconditions	A rule or ruleset has been modified to be enabled or disabled and the change is syncronized to affected sensors.

Use case	**Set rule threshold or suppression**
Primary actor	User
Purpose	Set or remove a threshold or suppression for a rule on one or more sensors.
Description	User sends a command with either thresholding/suppression parameters or removal of a threshold or suppression for a rule on one or more sensors. The system updates the database and distributes the change to the sensor(s).
Preconditions	Rules and sensor(s) must exist.
Postconditions	Rule tuning is saved in the central database and distributed to affected sensors.

Use case	**View rules**
Primary actor	User
Purpose	Display rules to the user.
Description	Rules are retrieved from the database, processed for GUI format and then delivered to the GUI.
Preconditions	Rules must exist.

Postconditions	The user is informed of the current status of the central database and the status of rules and rulesets enabled/disabled on sensors.

3.1.4 Detailed Use Cases

We also worked on some detailed use cases on our more complex system modules. Again, more use cases are available in appendix B.

Use case	**Central rule-database update**
Primary actor	Server
Purpose	Update the central rule database with the newest rules and revisions.
Description	An update can be triggered either by the user or the daemon. The system retrieves the files in question, checks what is new and what is not and then updates the database accordingly.
Preconditions	Depending on the request, this process requires a local file or the server must be configured with external source(s).
Postconditions	The central rule-database is up to date with the latest rules and revisions.
Triggers	Manual rule update, Automatic rule update, Write rule

Basic Flow:

1. Update is triggered.
2. Rule Downloading Module compares hash values from external sources with last update.
3. Updated rule files are downloaded from sources. The files are validated and passed to the Pre-Storage Processing Module.
4. The new files are parsed for rule SID and revision number.
5. A list of rule SID and revision numbers are gathered from the database and these are compared to identify which rules are new or modified.
6. New or changed rules are then parsed into objects and these objects are passed to the Database/Core Module.
7. The Database/Core Module stores the new or changed rule objects into the database. The user is notified that the update is complete and given a list of changes.

Extensions:

1. 1. Update is triggered by user.
   2. Update is triggered by daemon.
3. 1. The hash-values from all sources are identical with previously stored values. Database is up to date and the update process terminates. User is notified that database is up to date.
   2. The hash value from one or more sources is newer than previously stored values. Database must be updated, and the user is notified that the database is not up to date and an update process has begun.

Error handling:

1. The source and database checksum is different but the source rules are older than what is stored in the database: *The program can still continue, because if the SIDs already exist they will not be added again and if the revisions are older the rules will also not be updated.*
2. A ruleset has changed its name or one or more rules are moved to a different ruleset: *Based on a configuration file, the affected rules will either be a) automatically moved to the new ruleset or b) stay put in their original ruleset. The user will be notified and can choose to reverse or distribute the changes in either case.*

Use case	**Distribute rules**
Primary actor	Server
Purpose	Distribute rules from the central database to the sensors.
Description	After a command to synchronise is sent either by the user or the daemon, the server calculates and organizes the central database data and synchronizes this information with the affected sensors.
Preconditions	One or more rulesets or rules must be present in the central database. One or more sensors must be registered.
Postconditions	The rules on the sensor(s) are synchronized to reflect those in the central database that are enabled/disabled for those sensors.
Triggers	Manual rule distribution, Automatic rule distribution

Basic flow

1. Synchronize signal is sent.
2. The system calculates and organizes the data for each sensor affected by the synchronization.
3. The system initializes a synchronization protocol with the affected sensor or sensors, sending a list of what the sensor should have.
4. The sensor calculates diff between should have and has.
5. Compare diff.
6. Send update.
7. The sensor includes the new changes into its database and files and tells the central system that it is now up to date.
8. The system carries out maintenance to reflect the synchronization.
9. The user is notified that all sensors affected are now up to date.

Extensions

1. 1. User sends a command to synchronize one, more or all sensors
   2. The system reaches an interval for synchronizing all sensors.
   enumi4
3. 1. The sensor content is different from what it should have, asks central system to send what it needs.
   2. The sensor content is not different from what it should have, no further action is required. This status is sent to the central system.
3. 1. The system retrieves and sends what the sensor asks for.
   2. The system terminates the synchronization protocol with the sensor in question.

Error handling

1. The change may never be synced to the sensor due to errors: *There must be an error protocol that is followed in this scenario.*

3.1.5 Domain Model

3.2 Supplemental Requirements

3.2.1 Functionality

3.2.2 Usability

3.2.3 Reliability

3.2.4 Performance

3.2.5 Licensing

3.3 Constraints

3.3.1

3.3.2

3.3.3

4 DESIGN

Utfyllende med grunngitt valg av metoder, rammeverk etc. Hovedtrekk fra design av system beskrives. Fullstendig designdokument bør være vedlegg til prosjektrapporten, mens dette punktet omhandler diskusjon av sentrale forhold. Ulike alternative design som studentene vurderte bør beskrives og valg av alternativ begrunnes.

5 IMPLEMENTATION

This chapter will focus on the technical implementation of the software.

5.1 Frameworks

To simplify (db, web, code) this project uses the Django framework.

5.2 Concepts of Snort®

This section will give a brief introduction to the basic concepts of Snort®, and discuss these in relation to features made in this project.

5.2.1 Rules

We used to be fans of *modifiers*, but now we praise the hierarchy!

A rule is basically a set of options that define ways of detecting, classifying and reporting a potential attack or threat on the network[1]. There are several types of options, but only the general rule options are relevant for this project. The general rule options provide information about the rule[1] (i.e. revision number, alert message). Other types of options, such as payload options which are actively used in detection, are not relevant for the management of rules and are thus not considered in this project.

A rule can either be enabled or disabled in the detection system, meaning that the rule will only have an effect on detection when it is enabled. Since this project is dealing with multiple sensors connected to one central database, this functionality has been expanded so that users can keep a rule enabled on one sensor and disabled on another. In addition, the user can also set an enabled/disabled flag on rulesets.

5.2.2 Generators

5.2.3 Reference Types

5.2.4 Event and detection filters

Various filters can be applied to rules in order to control the amount and frequency of alerts triggered by a single rule.

Inline thresholds are currently supported, but this functionality is deprecated

Threshold deprecated. Detection filters.

If a filter occurs inline, this filter is regarded as a default filter and will be active on every sensor where the rule operates. A user can choose to override the default filter on a sensor by defining a custom filter to be used on the respective sensor.

To stop Snort®from adding duplicate filters, all inline filters are removed from the rulestrings. Any filters extracted from the rulestring are written as plain text (i.e. as they appear in the original rulestring) in the *filters* field of the rule revision object. This is to ensure that all the original rule options are stored meaning that no data is lost or discarded.

Filters are removed from the rulestring and transformed into event filter and detection filter objects.

[1] A rule actually consists of a header and options, rtfm.[1]

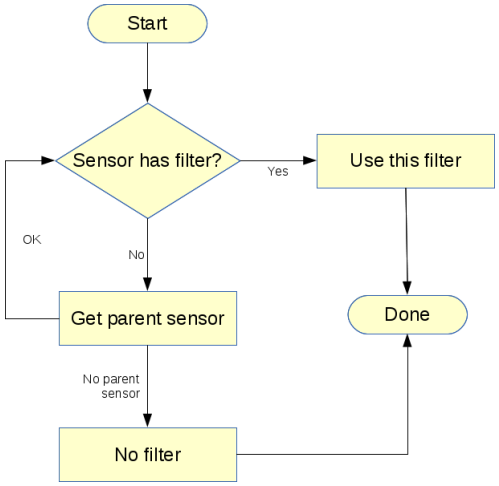


Figure 3: Flow

Filter type	Alerts
Event filter	
Limit	One for every event when *count* $\leq$ C in interval S
Threshold	One for every C events in interval S
Both	One per interval S when *count* $=$ C
Detection filter	One for every event when *count* $>$ C in interval S

Table 10: Effects of different filter types.

5.3 Core module

Lorem, ipsum?

5.4 Update module

The purpose of the Update module is to fetch and read rules and configuration files from external sources, and populate the database with this information. Effectiveness in updates is a key issue, and a number of deliberate optimizations for performing updates, with the goal of saving time by omitting unnecessary operations, are explained in the following sections.

The term Update is used to refer to the actual module, including all its classes and functions. To avoid confusion with the module, the term *update-package* is specifically used to refer to update data coming from external sources.

This section will define an example rule to be used throughout subsequent sections and the remainder of the chapter, to illustrate the lifespan of a rule in the system.

Word: rulestring.

5.4.1 Sources

A *source* is used by Update to determine where the update-package is located. Normally this is on a remote site, and the source contains the URL pointing to the location of the update-package. A source can be configured to update at regular intervals, or the user can run an update manually on a source. To make the update process more effective, Update makes use of an MD5 URL which points to a file containing the MD5 hash of the update-package. This hash is stored for every source (if available) and is compared to any pre-existing hash to verify if the system is up to date or not. Update will only download and process an update-package when it encounters a new hash.

5.4.2 Files

An update-package may contain various files depending on the source it comes from. This section will describe files supported by Snowman.

Rule files

A rule file is any file carrying the *.rules* file extension. These files contain raw rulestrings to be processed by Snort. A rulestring is basically a semicolon-separated list of rule options. Specific rule options used in this system are listed in table 11.

A badly formatted rulestring given to Snort will cause a crash, and Snort must be restarted.

Classification, Generator and Reference Configuration files

The classification files are identified by their name *classification.config* and contain rule classification strings that are used by Snort to categorize event data. A classification file will normally appear once per update-package, and rarely changes between updates[2].

Sid message files

5.4.3 Running an update

As an example scenario, consider a rule (name, figure reference, source, ruleset etc.). An update is triggered on the source at the predefined time

The update is cancelled if the source is already up to date; the MD5-hashes of the

[2]https://www.snort.org/vrt/snort-conf-configurations/

Data field	Description	Presence
sid	Unique identifier for a rule.	required
rev	Rule revision number.	required
msg	Notification text when rule triggers.	required
classtype	Connects the rule with a specific class of attacks.	required
gid	Generator ID to identify which subsystem generates the event where this rule fires.	optional
ruleset	The ruleset this rule belongs to.	optional
priority	Override default severity set by classtype.	optional
reference	Reference to external resource with information about this attack.	optional

Note: The *presence* column indicates whether a field is required or not by the parser in this system. Information is gathered from the Snort Users Manual [1](ch 3.4).

Table 11: Rule options used in this system.

latest update in the database, if any, and the latest update on the remote site are compared, and the process is terminated if these hashes are identical. Otherwise, the update is downloaded to a temporary directory and the update process continues. A file can be a tar-archive containing an arbitrary number of rulesets and configuration files, or it can be a plain text file containing Snort configuration (?).

The tar-archive is extracted, and the update module recursively parses each file in each folder. In most cases the update-package will contain only minor changes in relation to its predecessor, e.g. only one file is different in the latest package, and parsing all files that has not changed would therefore be a waste. To address this issue and save processing time, the name, path and MD5-hash of every file is stored in the database per source. Every time the update module parses a file, it first creates a hash of the file and compares this with the respective file's hash in the database. If the file is present in the database and the two hashes coincide, the file is considered *not new* and is skipped. The following sections will elaborate on the parsing of files based on file type.

parse files with regex.

5.4.4 Rule parsing

Some options can be defined both in the rulestring (inline) and in a separate file (standalone).

If any properties classified as *required* for a rule (table 11) are missing...

Non-existing rule sets, rule classes and generators are stored in the database

After the initial parsing of the rulestring, the parser checks if the rule already exists in the database, and if so whether the incoming revision number is higher than the latest revision of the pre-existing rule. The parsing will continue only in the event of a new rule or a newer revision of an existing rule.

rule moved to new ruleset.

Rule sets

A rule always belogs to a ruleset which can be specified in the raw rulestring. If the parser does not find this attribute in the rulestring, the filename is considered to be the ruleset name.

Threshold

A rule threshold defines a limit and/or threshold for the triggering of a specific rule, and can exist as an inline or standalone option. Rule thresholds are deprecated, and support for this functionality will cease in a future version of Snort®, where it is replaced by *detection filters* and *event filters* (see below) [1]. To maintain compatibility with current rules, Snowman supports reading inline and standalone threshold definitions.

Detection filters

Please explain..

Event filters

Please explain..

Suppression

Always standalone.

5.4.5 Classification, Generator and Reference Type parsing

These files contain a sequence of their respective entities (section 5.4.2), and are all parsed in the same manner. If a given entity is not already present in the database, it is created and stored. If the database already contains the entity, the existing entry in the database is overwritten.

Rule Message files

5.4.6 Logging

5.5 Graphical User Interface

The Graphical User Interface (GUI) is a template- and AJAX-based (desc/footnote?) Web application which uses Django's (desc) template system to implement the View part of the MVC-principle(ref). Each webpage is a template populated through Python-code before it is delivered to the Web server and ultimately the requesting Web browser.

Additional data that needs to be sent to a Web page, e.g. the user interacts with the GUI, is requested through JavaScript(desc) and AJAX-calls and then gets added to the already loaded DOM(desc) in the web browser.

5.5.1 Structure

The pages of the GUI is organized in the following structure:

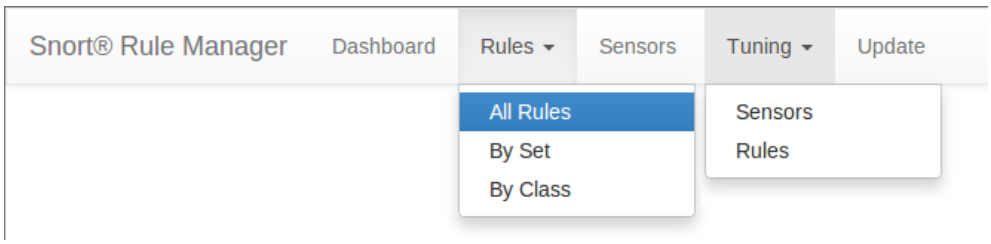


Figure 4: GUI Structure

5.5.2 Design

The GUI is designed with simplicity in mind, which is achieved by using a front-end framework called Bootstrap(ref). This framework provides easy-to-use and ready-made design elements for HTML and CSS(desc), which has been crucial for saving time and energy in the project. Consider the example job of creating a <table> element and spending hours setting up and tweaking the CSS for the said table element, we could just add <table class='**table table-bordered**'> and get the table formatted nicely with colors and fonts that are typographically matching.

5.5.3 Features

Content Prefetching

One of the main features of our GUI design is our usage of prefetching to speed up loading. Instead of loading absolutely everything into the web browsers DOM, like our predecessor BHTB did, we instead initially only load the first page or page elements. This significantly reduces loading times.

We then silently in the background load in any element the user might want to see next, through AJAX calls. When the user then interacts with the GUI, the next elements are shown to the user. The next-next elements are also loaded through AJAX calls so that they are ready.

This ensures that the user always has immidiate response from any interaction done with the GUI, while keeping loading times to a bare minimum.

5.5.4 Rules/All rules page

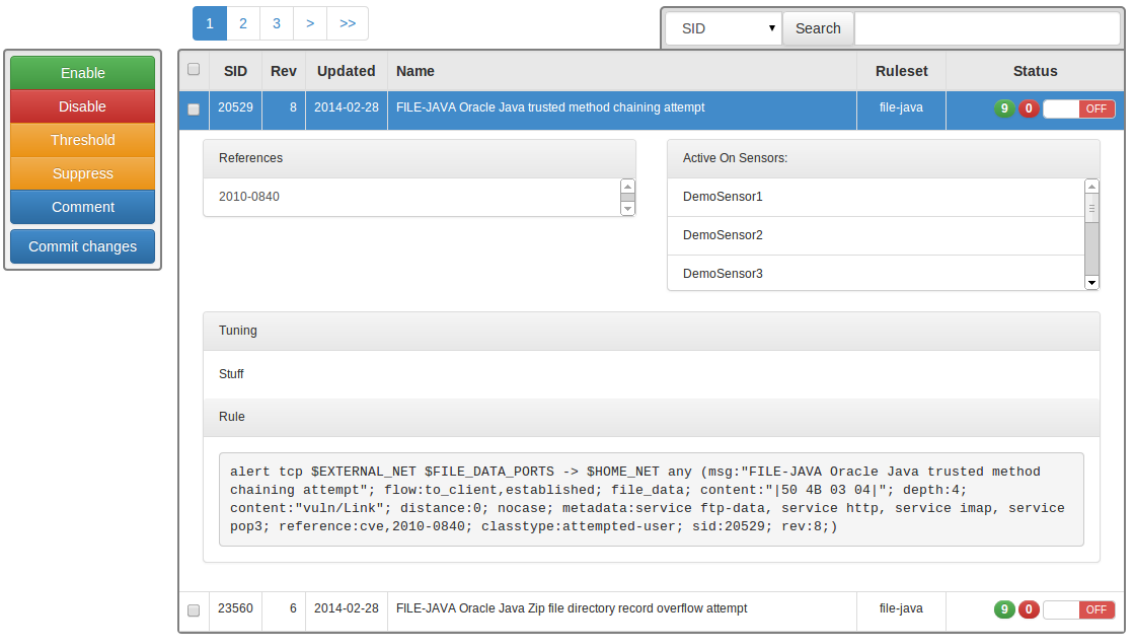


Figure 5: Rules/All rules page

This page serves as the complete list of all rules currently residing in the central database. Because the central database can literally contain hundreds of thousands of rules, this list is paginated. Pages are loaded using AJAX calls to prefetch them as needed.

Each row in the list can be clicked on, which expands an area containing additional information about the rule in question.

We anticipate that the main use of this page is to utilize the search function to find a specific rule or rules and then either retrieve information about them or tune them.

Rules can be tuned by checking the checkbox associated with the rule in question and then press the button that represent the desired function in the box to the left.

6 OPTIMIZATION

6.1 Update

7 SOFTWARE TESTING

Her beskrives hvordan opplegget rundt testing av applikasjonen har vært. Dette omfatter planlegging, valg av teststrategier, gjennomføring av tester og beskrivelser av sentrale feil funnet under testing, samt måten feil ble rettet opp på.

8 PERFORMANCE TESTING

Beskriv testenes formål og relater dette til kravspec.

The software is tested on three different architectures

To minimize error, each test was run five times where the mean value of the runs is considered the valid test result.

8.1 Environment

Beskriv testoppsett. maskinvare, nettverk, programvare, etc.

8.1.1 Test cases

Update processing

Q1: How fast can the software process an update?

The purpose of this question is to determine the average processing time per rule. Results are ranked from short to long processing time, where shorter is better.

Q2: How strong is the linear correlation between processing time and the size of an update?

This question rises from the requirements specification of Snowman (chapter 3), where it is stated that Snowman shall exhibit a linear increase in processing time in correlation to the number of rules in the update (where the number of rules depicts the update size). Results are ranked from strong correlation to no correlation, where stronger is better.

Q3: Does raising the ratio between files and rules increase processing time?

Updates can come with the rules either gathered in one file, or distributed over several files. This question is asked to examine the difference in processing time for these two scenarios. Results are ranked from no difference to big difference, where less difference is better.

Q4: Is the processing time shorter when the update contains no new rules?
Q5: Are the processing times for two updates identical when only one rule has changed between the updates?

This question seeks to determine how the software handles an update when there is only one change from the previous update. This is an important factor to consider, since an update usually contains only a small amount of changes from previous updates. Results are ranked from short processing time for second update to no difference in processing time, where shorter time is better.

An automatically generated update is used for Q1 through Q6

9 RESULTS

De faktiske observasjoner og data fra forsøk (f.eks. målinger) og undersøkelser (f.eks. utspørringer) presenteres på en oversiktlig måte ved tabeller og/eller diagrammer. Bruk skjønn for evt. plassering i vedlegg. Ved bruk av diagrammer skal det gis beskrivende tekst til grafer. Resultatene skal kort kommenteres, men ikke diskuteres. Viktige resultater gjengis kortfattet under ”Sammendrag”. En oversikt over tabeller og diagrammer med nummer og titler kan plasseres etter innholdsfortegnelsen.

Mean times calculated from five sequential test results. See appendix for a complete listing of results.

	No. of rules	No. of files	Rules per file	S1	S2	S3
Snortmanager	1	1	1	228.617	36.960	N/A
	10	1	10	265.275	37.569	N/A
	100	10	10	1251.275	159.113	N/A
	1000	1	1000	1836.396	228.044	N/A
	1000	100	10	9470.460	1360.276	N/A
	10000	1000	10	92428.555	13523.887	N/A
BHTB	1	1	1	228.617	36.960	N/A
	10	1	10	265.275	37.569	N/A
	100	10	10	1251.275	159.113	N/A
	1000	1	1000	1836.396	228.044	N/A
	1000	100	10	9470.460	1360.276	N/A
	10000	1000	10	92428.555	13523.887	N/A

Table 12: Test results for Snortmanager.

10 ANALYSIS OF RESULTS

Diskusjon rund resultatene. Det skal gjøres en evaluering av hvorvidtvalgt løsning. Avvik drøftes. det er samsvar mellom kravspesifikasjonen satt opp og

11 CONCLUSIONS

De slutninger som kan trekkes fra resultatene og deres diskusjon skal være logisk og nøkternt oppbygd. Etterrettelighet er vesentlig. Konklusjoner skal fremkomme klart og uten tvetydigheter. Eventuell usikkerhet påpekes. Grunnlaget for slik tvil skal redegjøres under kapittel 6a/6b. De viktigste konklusjoner blir presentert i kortfattet versjon under ”Sammendrag”. Prosjektgruppen bør ta stilling til hva de faglige resultatene kan brukes til, hva gruppen faglig har lært av dette arbeidet, hva som evt. kunne vært gjort annerledes ved en gjentagelse av utviklingsarbeidet, og hvordan eventuelt videre arbeid rundt samme eller beslektet tema bør gjøres. Gruppen bør også kommentere den subjektive opplevelse av hovedprosjektet, og hvordan gruppeprosessen har vært. Kort sagt: Egenevaluering av gruppearbeidet.

Bibliography

[1] Roesch, M. 2012. *Snort Users Manual 2.9.3*. The Snort Project.